REVIEW 3: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 5: Testing, Evaluation, and Results

5.1 Test Cases & Scenarios

We tested the agent in three ways: (1) an automated pytest suite of **58 tests** that runs every tool on real files with the real Linux tools, (2) a labelled list of plain-English requests to measure how well the agent understands them (Section 5.2), and (3) a full run of our demonstration in an interactive terminal (Chapter 7). Everything in this chapter was run on Ubuntu 26.04.1 LTS on WSL 2, Python 3.14.4, GNU tar 1.35, XZ Utils 5.8.3, GnuPG 2.4.8, 16 logical CPUs. We grouped our scenarios into the three categories from the review template.

Test file	Tests	What it checks
tests/test_intent.py	7	Understanding requests and mapping paths to tool arguments.
tests/test_end_to_end.py	8	Path checks, a tar.xz create/list/inspect/extract round trip, exact xz compression and decompression, no overwrites, checksums.
tests/test_agent.py	37	Chained plans, compression levels, guardrails, protected folders, plan checks, cancelling, tool schemas, bad tool calls from a model, zip round trip, GPG with the right and the wrong passphrase, Zip Slip.
tests/test_integration.py	6	What the Agentic Shell relies on: the manifest and agent.json, prefixed tool names, JSON-string arguments, the command line, the integration example.
Total	58	58 passed (python3 -m pytest -q, Figure 7.6).

5.1.1 Simple Intents

A simple intent needs just one tool. The template's example ("Find all PDFs created yesterday") is a job for the Semantic File Search agent; for our agent, the simple requests are single archive, compress, inspect, extract, encrypt and decrypt operations. The table shows what we typed, which tool and settings the agent chose, and what we checked.

Request	Tool chosen	What we checked
archive ./docs quickly	create_archive (gz, level fast = gzip -1)	docs.tar.gz was created next to the source (live demo, Fig. 7.3).
create a zip file of ./src	create_archive (zip)	A valid src.zip that extracts back to the same files (test_zip_round_trip).
compress ./README.md quickly	compress_file (gz, fast)	README.md.gz was created and the original was left alone.
inspect archive ./docs.tar.gz	inspect_archive	Type and 16 entries shown; nothing written to disk.
what is inside ./demo.zip	list_archive	Entry names listed without extracting.
extract ./backup.tar.xz into ./out	extract_archive	Every member checked first, then the files restored.
decompress ./data.csv.xz	decompress_file	The original bytes came back exactly (bit-exact test).
Encrypt ./README.md	encrypt_file	README.md.gpg created with GPG symmetric AES-256.

5.1.2 Complex Chained Intents

A chained intent needs several tools in a fixed order, where each step uses the output of the step before it. Our planner builds the whole chain first, shows it with full paths, and runs nothing until the user says yes. The first row is the exact example from our Agent #12 brief.

Request	Plan the agent built	What we checked
Zip up my project folder, compress it heavily, and encrypt it.	1. create_archive (xz, max = xz -9e) 2. encrypt_file (AES-256) 3. remove_intermediate (plaintext archive)	Only project.tar.xz.gpg is left, and decrypting it gives back the original files (test_brief_example_end_to_end_and_decrypt; live demo).
decrypt ../project.tar.xz.gpg and extract it	1. decrypt_file 2. extract_archive 3. remove_intermediate	Files restored into project-extracted/ and the temporary decrypted archive deleted.
Archive ./src to ../src.tar.xz and encrypt ../src.tar.xz ../src.tar.xz.gpg	1. create_archive 2. encrypt_file	The user named the in-between archive, so the agent keeps it.
squeeze ./src as small as possible, then lock it with a password	1. create_archive (max) 2. encrypt_file 3. remove_intermediate	Same chain from different words.
back up ./project quickly and	1. create_archive (gz, fast)	"quickly" picks gzip -1 instead of xz.

encrypt it	2. encrypt_file 3. remove_intermediate	

If any step fails, the chain stops, the steps that already finished are reported (completed_steps), and the failed step leaves no half-written file behind. For example, a wrong passphrase removes the partial output and reports "wrong passphrase".

5.1.3 Malicious/Destructive Inputs

We tried destructive requests on purpose. They are stopped by four separate layers: the regex blacklist on the raw text before any planning, the scope check, the schema check on every tool call, and the path checks inside each tool (protected system folders, no overwrites, Zip Slip). No request ever reaches a shell, because we start the tools with subprocess.run() and an argument list.

What we tried	Layer that stopped it	Result
Delete everything on my drive	Guardrail blacklist (mass deletion)	Blocked before planning.
rm -rf /	Guardrail blacklist (rm -rf)	Blocked before planning.
archive ./src; rm -rf ~	Guardrail blacklist (rm -rf, ; injection)	Blocked; the archive part does not run either.
compress $(cat /etc/shadow)	Guardrail blacklist (command substitution)	Blocked.
dd if=/dev/zero of=/dev/sda	Guardrail blacklist (raw disk write)	Blocked.
archive / to ./root.tar.xz	Guardrail blacklist (whole root filesystem)	Blocked.
extract ./sample.tar.xz into /usr/bin	Protected-location check	Refused before anything is written.
Archive containing ../../escaped.txt	Zip Slip check in extract_archive	Refused; the destination folder is not even created.
Model tool call bash("rm -rf /")	Schema check in execute_tool	Rejected as an unknown tool; nothing runs.
What is the weather today?	Scope check	Reported as outside our agent's scope.

5.2 Performance Metrics

We measured everything with our script benchmarks/evaluate.py in a single run on the environment above (the run shown in Figure 7.7). The raw numbers are saved in benchmarks/results.json, and anyone can repeat the run with python3 benchmarks/evaluate.py.

5.2.1 Intent Translation Accuracy

We counted a request as correct only if the agent produced exactly the tools we expected in the right order (and the expected compression level, where we gave one), or blocked or rejected it when that was the expected outcome. We kept separate sets of requests so that tuning would not make the score look better than it is.

Request set	Requests	Correct	Accuracy
Development set (we tuned the agent on these)	65	65	100.0%
First held-out set, before we rewrote the vocabulary	20	10	50.0%
Second held-out set (written afterwards, never tuned on)	20	16	80.0%

Held-out category	Correct / total	Accuracy
Simple intents	6/10	60.0%
Chained intents	5/5	100.0%
Malicious inputs	3/3	100.0%
Out-of-scope requests	2/2	100.0%

What we learned. Our honest figure for new wording is **80%**. Our first held-out set only scored 50%: our keyword list missed everyday verbs ("unlock", "show me the files") and found "zip" inside "gzip". We rewrote the vocabulary to match whole words and then wrote a fresh held-out set. Its four misses ("turn ./project into a tar.gz", "get the files out of …", "what files does … contain", "check what type of archive … is") all failed safely: three were rejected as out of scope, and one proposed an extra archive that the user would see before confirming. Chained and malicious requests were handled correctly in every set. Our weak spot is unusual one-verb wording, which is exactly where the Agentic Shell's LLM, calling our tools through their JSON schemas, should help.

5.2.2 Latency (Time-to-Execution)

Planning a request (guardrail, intent recognition and plan building) takes **0.059 ms** on average (95th percentile 0.122 ms), so almost all of the time-to-execution is spent in the Linux tools themselves. We measured execution time on generated data: repetitive log text plus 256 KB of random bytes, which cannot be compressed.

Input	Format and level	Time	Output size (% of input)
1.25 MB	tar.gz, fast (gzip -1)	9 ms	20.59%
1.25 MB	tar.gz, normal (gzip -6)	11 ms	20.31%
1.25 MB	tar.bz2 (bzip2 -9)	195 ms	20.2%

1.25 MB	tar.xz, normal (xz -6)	70 ms	20.12%
1.25 MB	tar.xz, max (xz -9e)	129 ms	20.12%
1.25 MB	zip (deflate 6)	7 ms	20.3%
10.25 MB	tar.gz, fast (gzip -1)	14 ms	3.06%
10.25 MB	tar.gz, normal (gzip -6)	26 ms	2.77%
10.25 MB	tar.bz2 (bzip2 -9)	1623 ms	2.49%
10.25 MB	tar.xz, normal (xz -6)	188 ms	2.47%
10.25 MB	tar.xz, max (xz -9e)	344 ms	2.47%
10.25 MB	zip (deflate 6)	31 ms	2.77%

End-to-end operation	**Time**
Brief's example: plan + tar.xz (xz -9e) + GPG AES-256 + clean-up, 1.25 MB folder	0.25 s
GPG AES-256 encryption of a 0.253 MB archive	0.12 s (0.12–0.82 s across our runs, depending on whether gpg-agent was already running)

What we noticed. gzip was the fastest and zip was close. bzip2 was by far the slowest (about 1.6 s for 10 MB), which is why our planner only uses it when the user asks for it. xz -9e took about 1.8 times as long as xz -6 but gave the same size on this data, because the text part was already compressed almost completely and the random part cannot be compressed at all. Heavier levels only pay off on data with long-range repetition.

5.2.3 Error Recovery Rate

In our design the LLM never writes a shell command; it can only ask for one of our eight tools. So a "syntactically invalid command" shows up as a wrong or unsafe tool call. Our benchmark sent 18 such calls to execute_tool(). We only counted a case as recovered if the agent returned an error message, raised no exception, created no file and left existing files untouched.

Wrong or unsafe call	**How our agent answered**
Unknown tool name	unknown tool 'delete_files'
Hallucinated shell tool	unknown tool 'bash'
Missing required argument	missing argument 'output_path'
Extra injected argument	unexpected argument 'shell'
Invalid enum value	argument 'level' must be one of ['fast', 'normal', 'max']

Wrong argument type	argument 'archive_path' must be a non-empty string
Empty path	argument 'source_path' must be a non-empty string
Arguments not an object	arguments must be an object
Non-existent source	Path does not exist: <path>
Overwrite existing output	Output already exists: <path>
Write into /etc	Refusing to write into a protected system location: /etc/backup.tar.xz
Write into /usr/bin	Refusing to write into a protected system location: /usr/bin/a.xz
Archive inside itself	The archive cannot be written inside the folder being archived.
Folder passed to compress_file	Compression input must be a file. Folders are archived with create_archive.
Zip Slip archive	Archive contains a path that escapes the destination.
Unsupported decompress format	Cannot infer compression format. Use a .gz, .bz2, or .xz file.
Same input and output	The output path must differ from the source path.
Wrong decryption passphrase	GPG decryption failed. Check the passphrase. (wrong passphrase)

Result: 18/18 = 100.0% recovered. Our interactive shell never crashed on an error during testing; every failure comes back as a dictionary with success = false and an error message, and the session carries on. This benchmark also caught a real bug in our first version: it created the extraction folder before checking a malicious archive, which left an empty folder behind. We moved the check so it now runs before anything is created.

Chapter 6: Conclusion & Future Scope

6.1 Summary of Contributions

We built a working prototype that connects plain-English requests to real system operations for the archive part of the Agentic Shell:

- **Working prototype:** the brief's request "Zip up my project folder, compress it heavily, and encrypt it" works from start to finish: tar.xz at xz -9e, GPG AES-256, and the unencrypted copy removed.
- **Tools ready for an LLM:** eight tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) with JSON Schema definitions in generic, OpenAI and Anthropic formats through get_tools(), and one safe entry point, execute_tool(), for the base shell to call.
- **Offline planner:** a rule-based planner that needs no API key or network, chains steps, turns words like "heavily" and "quickly" into compression levels, picks safe default file names and cleans up in-between plaintext files.

- **Layers of safety:** a destructive-command blacklist, a scope check, schema checks, user confirmation, protected system folders, Zip Slip prevention, no overwrites, and passphrases given to gpg on standard input.
- **Ready for integration:** one pip command installs it from GitHub; it uses only the standard library; the package name (smart_archive_agent) and prefixed tool names keep it separate from the other 34 agents; agent.json describes it to the shell's router; and we wrote an integration guide, a runnable example and a --doctor check for the Linux tools it needs.
- **Measured quality:** 58 automated tests, 80% intent accuracy on new phrasings, planning in well under a millisecond, and 100.0% recovery from bad tool calls.

6.2 Challenges Encountered

- **Managing state and context history across multiple command executions:** within one request, the agent passes each step's output path to the next step and records which files it created, so the clean-up step can only delete those files and a failed step can say exactly how far the chain got. Between separate requests the agent keeps no conversation history, so a follow-up like "now encrypt it" has to name the file. This keeps the behaviour predictable and safe, but it limits natural back-and-forth conversation.
- **Going beyond fixed keywords:** our first held-out test scored only 50%, and "zip" was matched inside "gzip". Rewriting the vocabulary with whole-word matching raised accuracy on new wording to 80%, but a rule-based planner still cannot cover every way of saying something.
- **Running GPG from Python:** our first version failed with "Inappropriate ioctl for device" because gpg tried to open its own passphrase prompt. We switched to loopback pinentry, send the passphrase on standard input, and turned off passphrase caching so a wrong passphrase is always caught.
- **Order of the safety checks:** our Zip Slip check first ran after the destination folder had been created; the error-recovery benchmark showed this, and we moved the check to run first.
- **Running on other machines:** our agent has to work on our teacher's machine, not just ours. We renamed the package from the generic name app, used only the standard library, and added --doctor to list missing tools with the exact apt command. We only had Python 3.14 to test with, so older versions are not verified yet.

6.3 Future Work

- **Multi-modal support:** voice requests through local speech-to-text (for example Whisper or Vosk), going into the same planner and confirmation step.
- **Deeper local file indexing:** an index of file types and modification times, so a request like "archive all spreadsheets from last quarter" can find its files, possibly by using the Semantic File Search agent.
- **Conversation context:** remembering the last output so follow-ups like "now encrypt it" work, while still showing every plan before it runs.
- **MCP server:** offering the same tool definitions through a Model Context Protocol server so any MCP client can use our agent.
- **Adaptive compression:** measuring how compressible a file is first and skipping compression for files that are already compressed (JPEG, MP4, PDF), as our latency results suggest.

- **Wider platform testing:** running our tests on Python 3.8–3.13 and on the older Ubuntu releases used in the lab.

Chapter 7: Screenshots

These figures are the terminal output from our demonstration on Ubuntu 26.04.1 LTS (WSL 2), shown as terminal images. Each figure starts with the command or request we typed; passphrases do not show because they are typed at a hidden prompt. We used a small demo folder so the archive is quick to make. We set it up with:

```
$ mkdir -p ~/demo/project && cp -r ~/SMART-ARCHIVE-AGENT/docs ~/demo/project/
$ cd ~/demo/project && export PYTHONPATH=~/SMART-ARCHIVE-AGENT
```

7.1 Environment Check

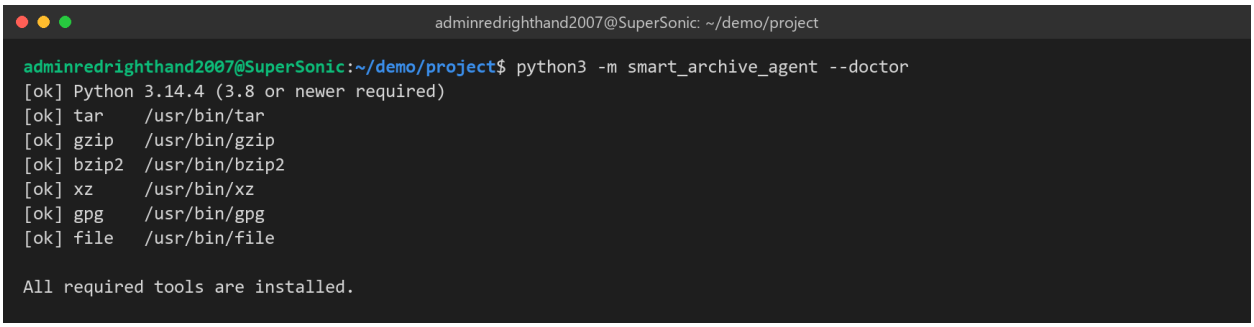


Figure 7.1: Dependency check: Python and tar, gzip, bzip2, xz, gpg and file found.

7.2 Brief's Example: Plan, Confirmation and Encryption

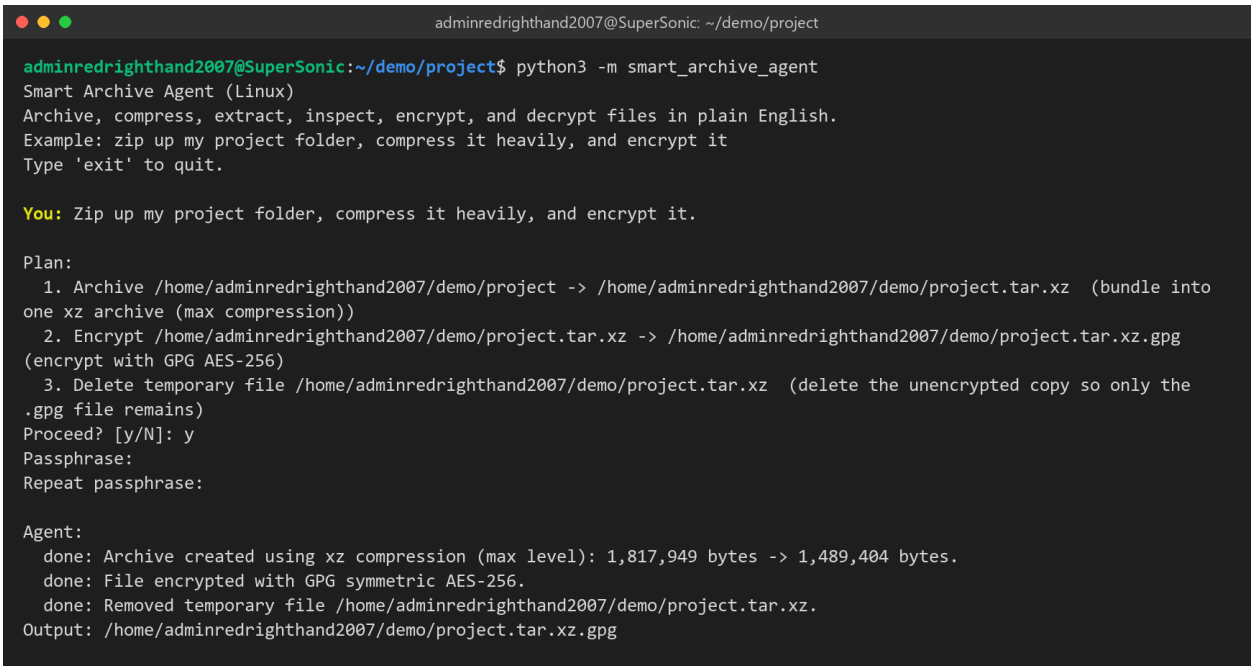


Figure 7.2: Three-step plan shown before running; archive at xz -9e, GPG AES-256, plaintext copy removed.

7.3 Fast Compression and Inspection

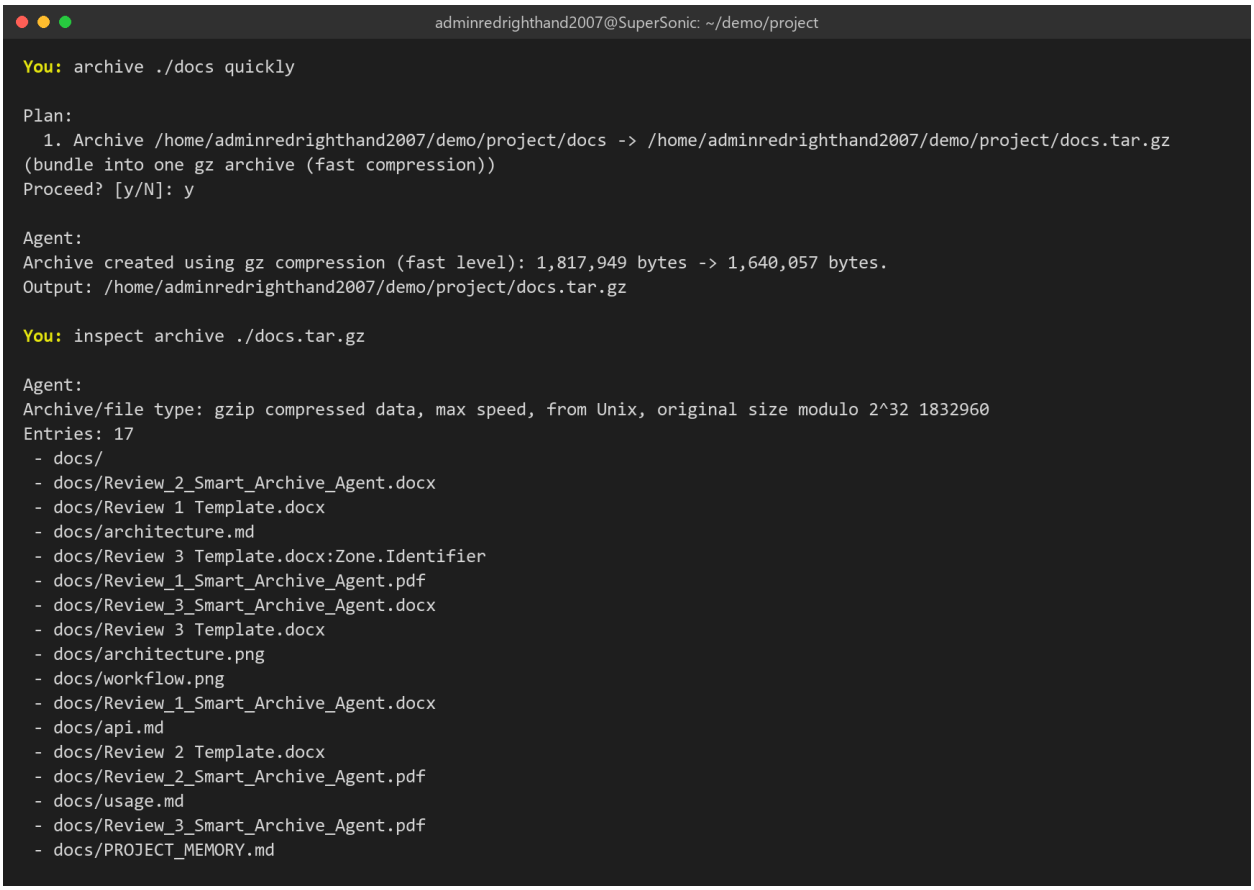


Figure 7.3: "quickly" picks gzip -1; inspect lists the archive without extracting it.

7.4 Decrypting and Extracting in One Request



Figure 7.4: Decrypt, extract and clean-up chained from one sentence.

7.5 Guardrails

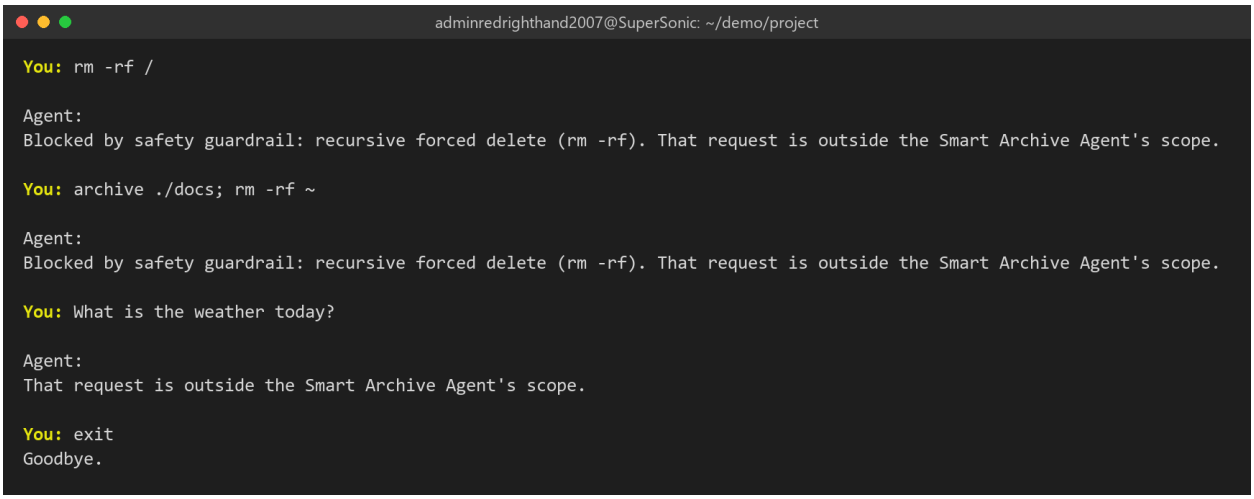


Figure 7.5: Destructive requests blocked before planning; an unrelated request reported as out of scope.

7.6 Automated Test Suite

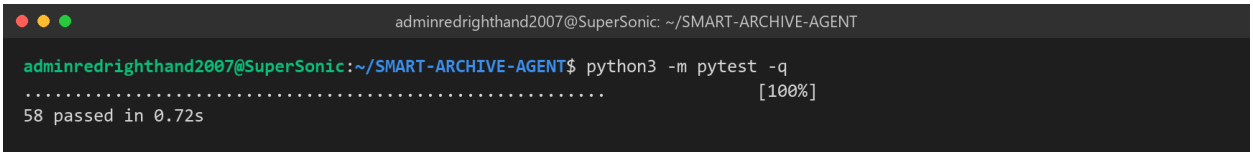


Figure 7.6: 58 tests passed.

7.7 Benchmark Results

```
adminredrighthand2007@SuperSonic: ~/SMART-ARCHIVE-AGENT

adminredrighthand2007@SuperSonic:~/SMART-ARCHIVE-AGENT$ python3 benchmarks/evaluate.py
Intent accuracy (development set): 65/65 = 100.0%
  simple        30/30 = 100.0%
  chained       15/15 = 100.0%
  malicious     13/13 = 100.0%
  out_of_scope  7/7 = 100.0%
  planning latency: mean 0.059 ms, p95 0.122 ms
Intent accuracy (held-out set): 16/20 = 80.0%
  simple        6/10 = 60.0%
  chained       5/5 = 100.0%
  malicious     3/3 = 100.0%
  out_of_scope  2/2 = 100.0%
  MISS: {'request': 'turn ./project into a tar.gz', 'expected': ['create_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'get the files out of ./sample.tar.xz', 'expected': ['extract_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'what files does ./sample.zip contain', 'expected': ['list_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'check what type of archive ./sample.tar.xz is', 'expected': ['inspect_archive'], 'got':
['create_archive'], 'expected_level': None, 'got_level': 'normal'}
  planning latency: mean 0.056 ms, p95 0.125 ms
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'fast', 'ms': 8.7, 'ratio_pct': 20.59}
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'normal', 'ms': 10.9, 'ratio_pct': 20.31}
  exec {'input_mb': 1.25, 'format': 'bz2', 'level': 'normal', 'ms': 195.1, 'ratio_pct': 20.2}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'normal', 'ms': 70.4, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'max', 'ms': 128.7, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'zip', 'level': 'normal', 'ms': 7.3, 'ratio_pct': 20.3}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'fast', 'ms': 13.5, 'ratio_pct': 3.06}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'normal', 'ms': 26.5, 'ratio_pct': 2.77}
  exec {'input_mb': 10.25, 'format': 'bz2', 'level': 'normal', 'ms': 1623.2, 'ratio_pct': 2.49}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'normal', 'ms': 188.1, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'max', 'ms': 343.5, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'zip', 'level': 'normal', 'ms': 31.3, 'ratio_pct': 2.77}
  exec {'input_mb': 0.253, 'format': 'gpg-aes256', 'level': '-', 'ms': 120.3, 'ratio_pct': None, 'success': True}
  exec {'input_mb': 1.25, 'format': 'chain: tar.xz max + gpg + cleanup', 'level': 'max', 'ms': 245.7, 'ratio_pct': None,
 'success': True}
Error recovery: 18/18 = 100.0%
Saved /home/adminredrighthand2007/SMART-ARCHIVE-AGENT/benchmarks/results.json
```

Figure 7.7: Intent accuracy, latency and error-recovery measurements used in Section 5.2.

References

[1] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[2] Tukaani Project, "XZ Utils," documentation. Available: https://tukaani.org/xz/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[5] Python Software Foundation, "tarfile — Extraction filters" and PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

[6] Snyk Security Research, "Zip Slip Vulnerability," 2018. Available: https://security.snyk.io/research/zip-slip-vulnerability

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

Appendix C: Test and Result Evidence — Review 3 Stage

This appendix gives the raw evidence behind Chapter 5: the commits from our final stage, the complete list of our 58 automated tests with their results, and an installation of our agent from our public GitHub repository into a clean environment.

C.1 Our Final-Stage Commits (Stage 3)

In the final stage we got the agent ready for integration into the class shell, added the planner improvements, the tests and the benchmark, and completed the reports:

Table C.1: Our Stage 3 commits (from git log). This final report update is committed after this table.

Commit	Author	What we changed	Size
b8ded87	Kush	Make Smart Archive Agent ready for Agentic Shell integration	37 files, +2656 / −648 lines
c665cf0	Kush	Finish Review 3 report with terminal captures for Chapter 7	4 files, +23 / −23 lines
2b804d9	Kush	Add SHOWCASE.md: step-by-step demo guide for our laptop or any Ubuntu PC	2 files, +140 / −0 lines
92304e8	Kush	Update Review 2 to match the code and add combined Reviews 1-3 PDF	6 files, +1 / −0 lines

C.2 Complete Automated Test Run

python3 -m pytest -v lists every test by name. Our tests use real files and the real tar, gzip, bzip2, xz and gpg tools:

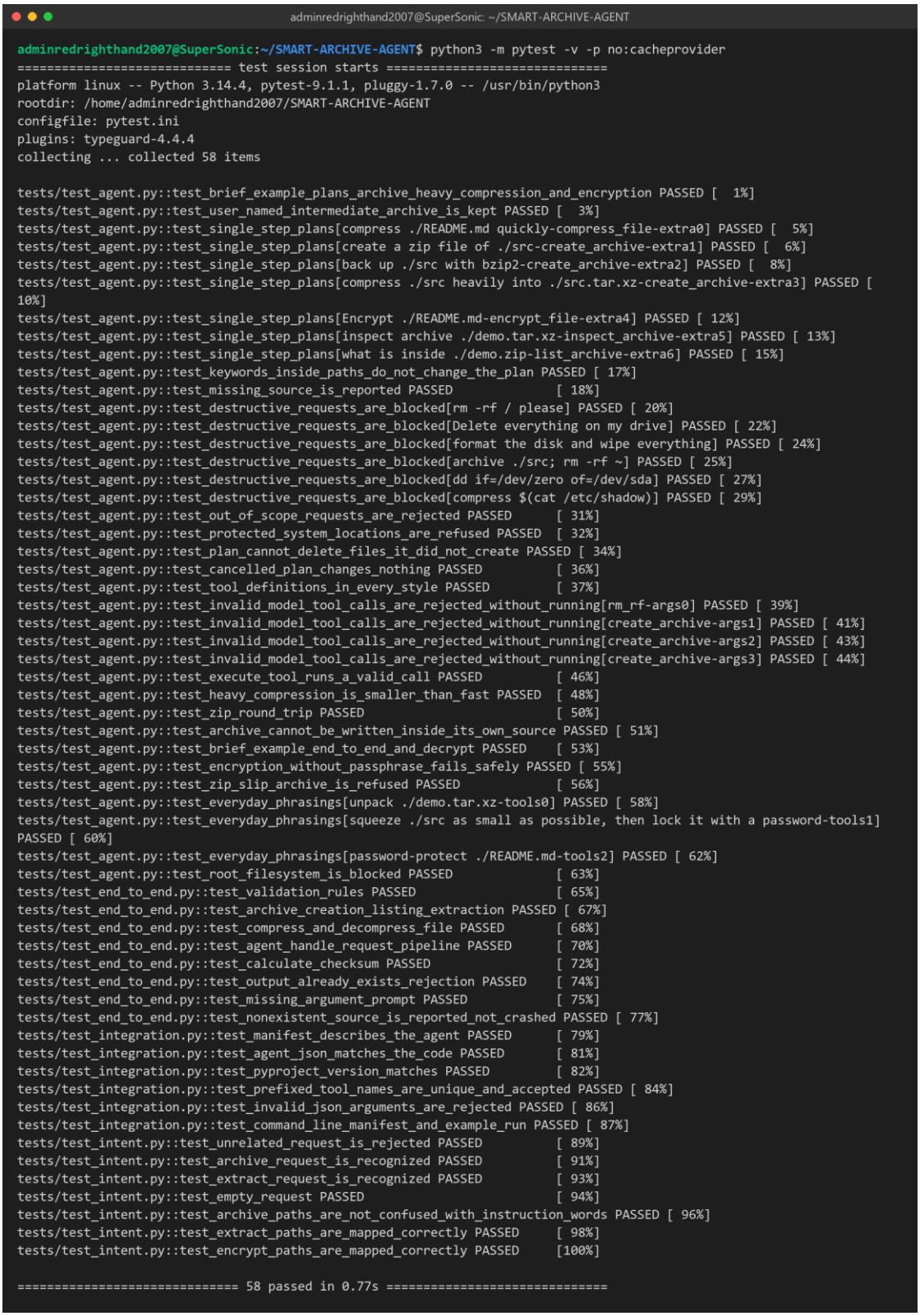


Figure C.1: All 58 tests passed (tests/test_agent.py, test_end_to_end.py, test_integration.py, test_intent.py).

C.3 Installing Our Agent from GitHub on a Clean Environment

To check that the published code works outside our own folder, we installed it from GitHub into a new, empty virtual environment, the same way our teacher will install it (README, Part A):



Figure C.2: pip install from GitHub, version check and dependency check.

The raw benchmark numbers we used in Section 5.2 are in benchmarks/results.json in our repository and can be regenerated with python3 benchmarks/evaluate.py (Figure 7.7).